

ChaCha vs AES vs RSA

Sreejit Maity

4th August 2026

1 Introduction

Modern cryptographic engineering relies on three core primitives to secure digital infrastructures: stream ciphers, block ciphers, and public-key asymmetric algorithms. This paper evaluates the structural characteristics and execution performance profiles of three defining standards: **ChaCha20**, **AES-256**, and **RSA-2048**.

2 Algorithmic Classifications and Structural Designs

The chosen primitives differ fundamentally in mathematical construction, data processing techniques, and computational complexities.

- **ChaCha20 (Symmetric Stream Cipher):** Designed by Daniel J. Bernstein, ChaCha20 updates a 512-bit internal state matrix through an iterative sequence of 20 computational rounds. It relies strictly on **ARX** operations (Addition, Rotation, and XOR). Because it avoids memory lookup tables entirely, it is naturally immune to cache-timing side-channel attacks and offers high performance in software-only environments.
- **AES-256 (Symmetric Block Cipher):** Operating as a Substitution-Permutation Network (SPN), AES-256 encrypts static 128-bit blocks over 14 execution loops. It relies on byte-level substitution (*S-Boxes*) and linear diffusion steps (*MixColumns*). While highly performant on hardware equipped with dedicated instruction extensions (AES-NI), raw software implementations face strict cache-timing attack constraints.
- **RSA-2048 (Asymmetric Cipher):** Built on the mathematical hardness of the integer factorization problem, RSA manages asymmetric key pairs. It utilizes a public exponent (e) for fast encryption and a large private exponent (d) for decryption via modular exponentiation:

$$C = M^e \pmod{N} \quad \text{and} \quad M = C^d \pmod{N} \quad (1)$$

Operating on 2048-bit variables, it demands significantly higher computation thresholds than its symmetric counterparts.

3 C Benchmarking Simulation Framework

The following complete C code evaluates the structural execution cycles of these algorithms. It measures processing overhead on a single architecture using low-level inline x86 assembly directives (`rdtsc`).

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <string.h>
4
```

```

5 // Inline hardware hook to track true CPU Clock Cycles via Assembly
6 static inline uint64_t get_clock_cycles() {
7     unsigned int lo, hi;
8     __asm__ __volatile__ ("rdtsc" : "=a" (lo), "=d" (hi));
9     return ((uint64_t)hi << 32) | lo;
10 }
11
12 #define ROTL(v, n) (((v) << (n)) | ((v) >> (32 - (n))))
13 #define CHACHA_QUARTERROUND(a, b, c, d) \
14     a += b; d ^= a; d = ROTL(d, 16); \
15     c += d; b ^= c; b = ROTL(b, 12); \
16     a += b; d ^= a; d = ROTL(d, 8); \
17     c += d; b ^= c; b = ROTL(b, 7);
18
19 // 1. Simulating ChaCha20 Core Block Loop
20 void run_chacha20_block(uint64_t *cycles) {
21     uint32_t state[16] = {0};
22     uint64_t start = get_clock_cycles();
23
24     for (int i = 0; i < 10; i++) { // 20 rounds (2 rounds per loop)
25         CHACHA_QUARTERROUND(state[0], state[4], state[8], state[12]);
26         CHACHA_QUARTERROUND(state[1], state[5], state[9], state[13]);
27         CHACHA_QUARTERROUND(state[2], state[6], state[10], state[14]);
28         CHACHA_QUARTERROUND(state[3], state[7], state[11], state[15]);
29         CHACHA_QUARTERROUND(state[0], state[5], state[10], state[15]);
30         CHACHA_QUARTERROUND(state[1], state[6], state[11], state[12]);
31         CHACHA_QUARTERROUND(state[2], state[7], state[8], state[13]);
32         CHACHA_QUARTERROUND(state[3], state[4], state[9], state[14]);
33     }
34
35     *cycles = get_clock_cycles() - start;
36 }
37
38 // 2. Simulating AES-256 State Round Loop
39 void run_aes256_block(uint64_t *cycles) {
40     uint8_t state[16] = {0};
41     uint64_t start = get_clock_cycles();
42
43     for (int r = 0; r < 14; r++) { // 14 Rounds for AES-256
44         for(int i = 0; i < 16; i++) { state[i] ^= 0x5A; } // Simulated
45         AddRoundKey/SubBytes
46     }
47
48     *cycles = get_clock_cycles() - start;
49 }
50
51 // 3. Simulating RSA-2048 Modular Multiplication Loop (Basic Montgomery Step)
52 void run_rsa2048_step(uint64_t *cycles) {
53     uint64_t dummy_accumulator = 1;
54     uint64_t pseudo_prime = 0xFFFFFFFFFFFFFFFF;
55     uint64_t start = get_clock_cycles();
56
57     // RSA decryption involves complex exponentiation.
58     // This loop simulates a fraction of the necessary 2048-bit multiplication
59     steps.
60     for (int i = 0; i < 2048; i++) {
61         dummy_accumulator = (dummy_accumulator * 3) % pseudo_prime;
62     }
63
64     *cycles = get_clock_cycles() - start;
65 }
66
67 int main() {

```

```

66     uint64_t chacha_cyc, aes_cyc, rsa_cyc;
67
68     printf("--- Initiating Cryptographic Primitive Profiling ---\n\n");
69
70     run_chacha20_block(&chacha_cyc);
71     run_aes256_block(&aes_cyc);
72     run_rsa2048_step(&rsa_cyc);
73
74     printf("=====\n");
75     printf(" Primitive      Category      Data Size      Sim. Cycles  \n");
76     printf("=====\n");
77     printf(" ChaCha20      Stream Cipher  64 Bytes      %llu\n", (unsigned long
78         long)chacha_cyc);
79     printf(" AES-256      Block Cipher   16 Bytes      %llu\n", (unsigned long
80         long)aes_cyc);
81     printf(" RSA-2048      Asymmetric    256 Bytes      %llu\n", (unsigned long
82         long)rsa_cyc);
83     printf("=====\n");
84
85     return 0;
86 }

```

Listing 1: Single C Program for Comparative Primitives Benchmarking

4 Architectural Comparisons and Computational Overhead

Analyzing operational execution paths highlights structural discrepancies in latency profiles across software implementations.

Asymmetric protocols like RSA-2048 are several orders of magnitude heavier than symmetric options. They scale quadratically with key length due to multi-precision math processing limits. As a result, systems generally utilize asymmetric protocols exclusively during initial channel handshakes to safely negotiate lightweight symmetric session keys.

Table 1: Cryptographic Paradigm and Structural Performance Analysis

Primitive	Type	Bits	Core Operations	Software Speed Profile
ChaCha20	Symmetric Stream	256	ARX (Add/Rotate/XOR)	Extremely Fast (Consistent)
AES-256	Symmetric Block	256	SPN (S-Box Tables)	Fast (Hardware-Dependent)
RSA-2048	Asymmetric	2048	BigInt Modulo Exponentiation	Very Slow (Heavy Overhead)

5 Conclusion

Maximizing computational throughput requires matching algorithms to specific systems architectures. ChaCha20 provides optimal software-level efficiency for mobile environments lacking native co-processors. AES-256 delivers exceptional security and speed across desktop and server configurations when hardware-accelerated instructions are available. Finally, RSA-2048 serves as a secure, foundational baseline for digital signature validation and secure asymmetrical key exchanges.

I have copied the whole code from Gemini and the detailing is done by the help of Wikipedia, Chat GPT and various websites. Latex format and quick notations are run on Gemini.